Habib University
shaping futures

CS201 – Data Structures II

Lecture 11

Dr. Muhammad Mobeen Movania

(Content has been taken from lecture slides of
Dr. Syeda Saleha Raza)

Outline

- Memory management and memory hierarchy
- Multiway Search Trees
- B-trees
- (2,4) Trees
- Operations
  - Creation
  - Add
  - Delete

Memory management

- Efficiency of data structures and algorithms is measured in two ways:
  - **Amount of time** the data structure or algorithm takes
  - **Amount of space** the data structure or algorithm takes

- So far we have been focusing on measuring the efficiency of data structures and algorithms in terms of the **amount of time**

- In practice, the performance of a computer system is also greatly impacted by the management of the computer's **memory**

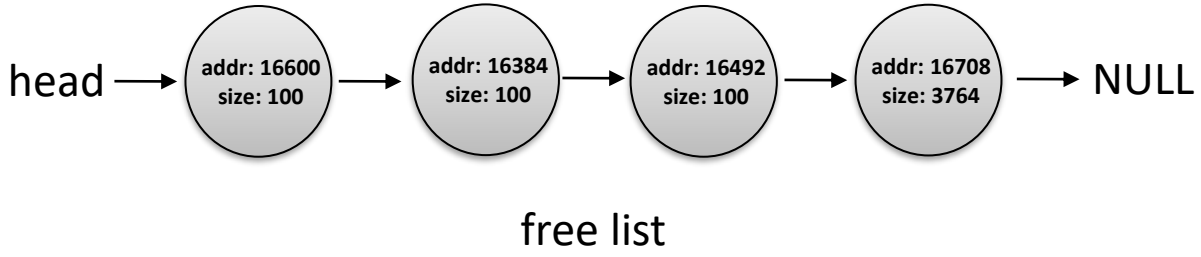
Organization of Memory

- Memory is organized in terms of bytes and their combinations
  - The simplest unit is 1 bit
  - 4 bits = 1 nibble
  - 8 bits = 2 nibbles = 1 byte
  - 2 bytes = 16 bits = 4 nibbles = 1 word
  - 2 words = 4 bytes = 32 bits = 8 nibbles = 1 double word or 1 dword
- Compute memory is organized into a sequence of words
  - Each memory word is associated with a number called its **address** in memory
- Available memory words are organized into larger contiguous units of fixed or variable size called **blocks**
  - A linkedlist is maintained in memory called a **freelist** which contains a list of available contiguous allocations (their addresses and their allocation sizes)

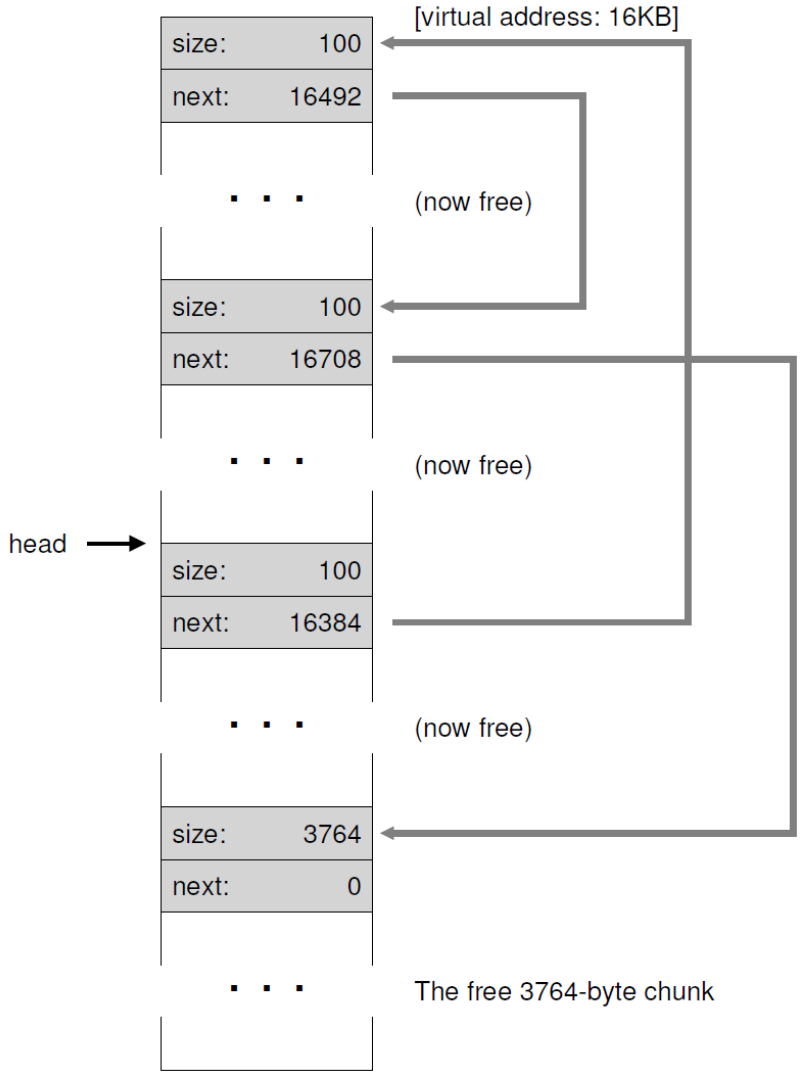
3.3 Data Organization (plantation-productions.com)

Example: Deletion of all memory allocations

- Due to several deletions, memory is fragmented storing several small contiguous allocations
- Memory coalescing helps to remove fragmentation

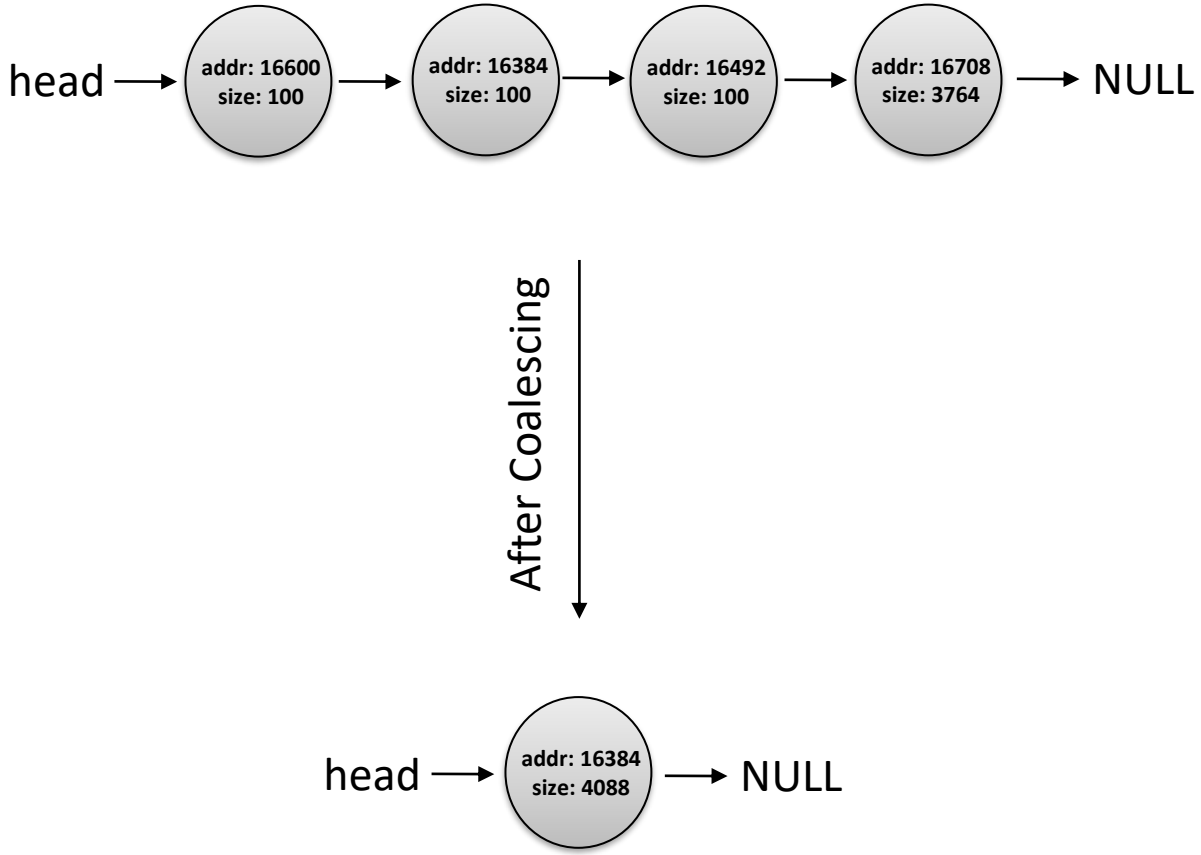


free list



Example: Deletion of all memory allocations

- Coalescing can help remove fragmentation in the freelist



Block allocation strategies (policies)

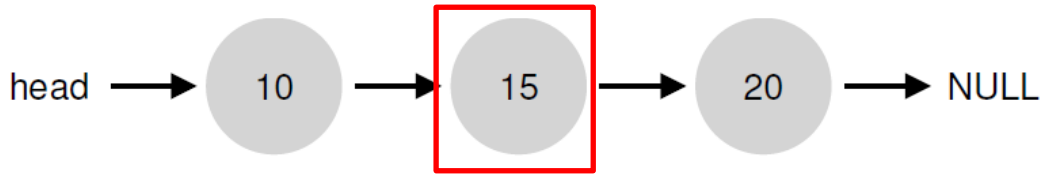
- Best fit
  - Returns the smallest block that satisfies our request
- Worst fit
  - Returns the biggest block that satisfies our request
- First fit
  - Returns the first block that satisfies our request
- Next fit
  - Returns the next block (from the last block allocated) that satisfies our request

Examples (15 bytes alloc. req.)

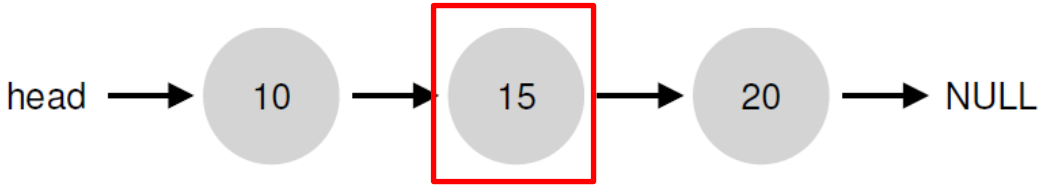
- Original list

- Best fit

- Worst fit



- First fit

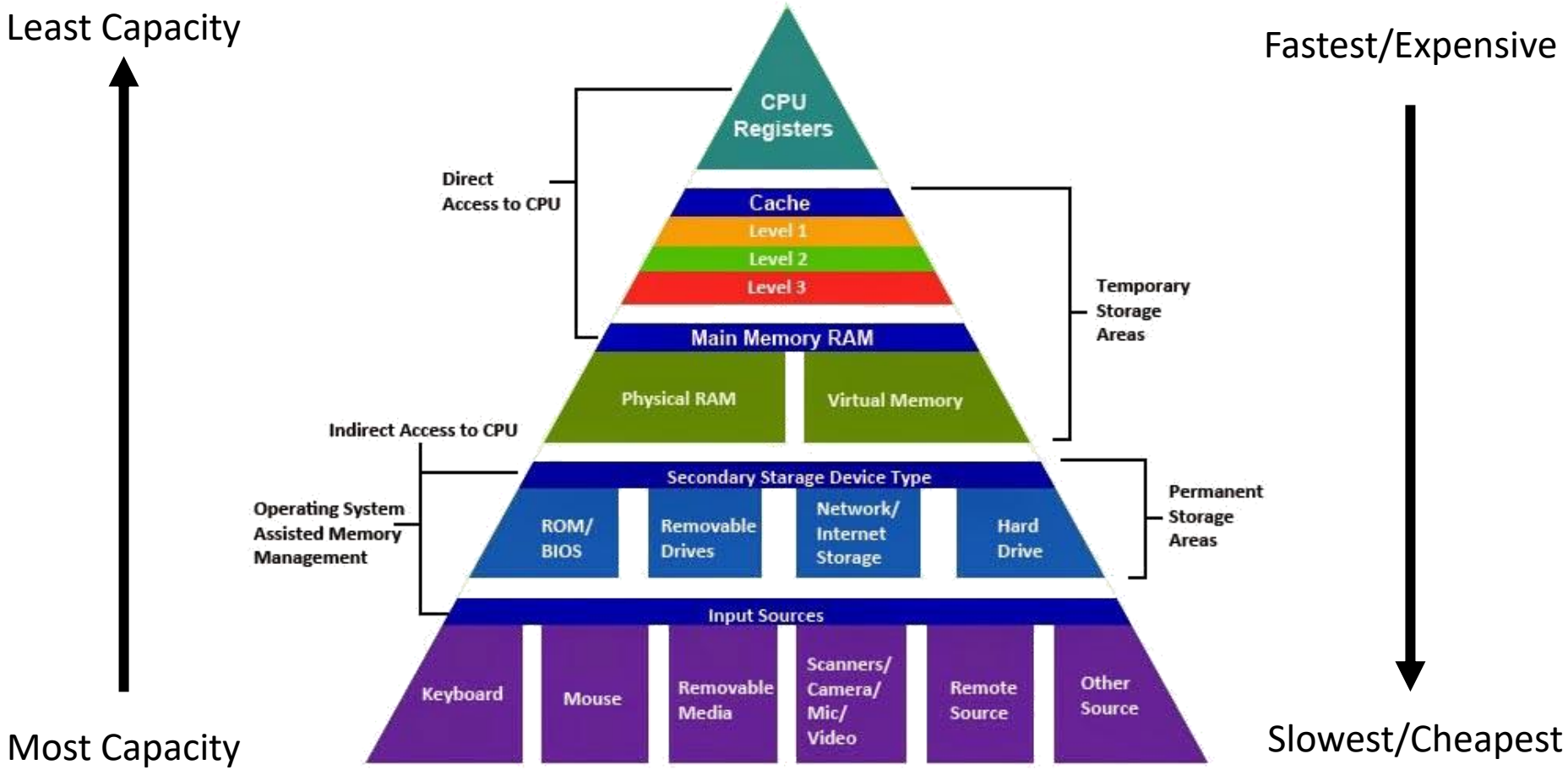


Garbage collection

- In C/C++, there are two types of memory allocations
  - Static memory allocation
    - Allocated at compile time (for e.g. automatic local variables, global and static variables) are automatically reclaimed by the compiler runtime when the variable goes out of scope
  - Dynamic memory allocation
    - Allocated using **new / malloc** must be reclaimed by the programmer by explicitly calling **delete / free**
    - Failing to do so results in **memory leak**
- Modern C++ and languages like python, java, C# provide automatic memory reclaiming called **garbage collection (**through **reference counting** or **mark-sweep algorithm**)
  - It looks at the number of active (**live**) objects having direct or indirect reference/s
  - If an object has no indirect or direct reference, it is deleted from the memory

Memory Hierarchy

Least Capacity



Most Capacity

Fastest/Expensive

Slowest/Cheapest

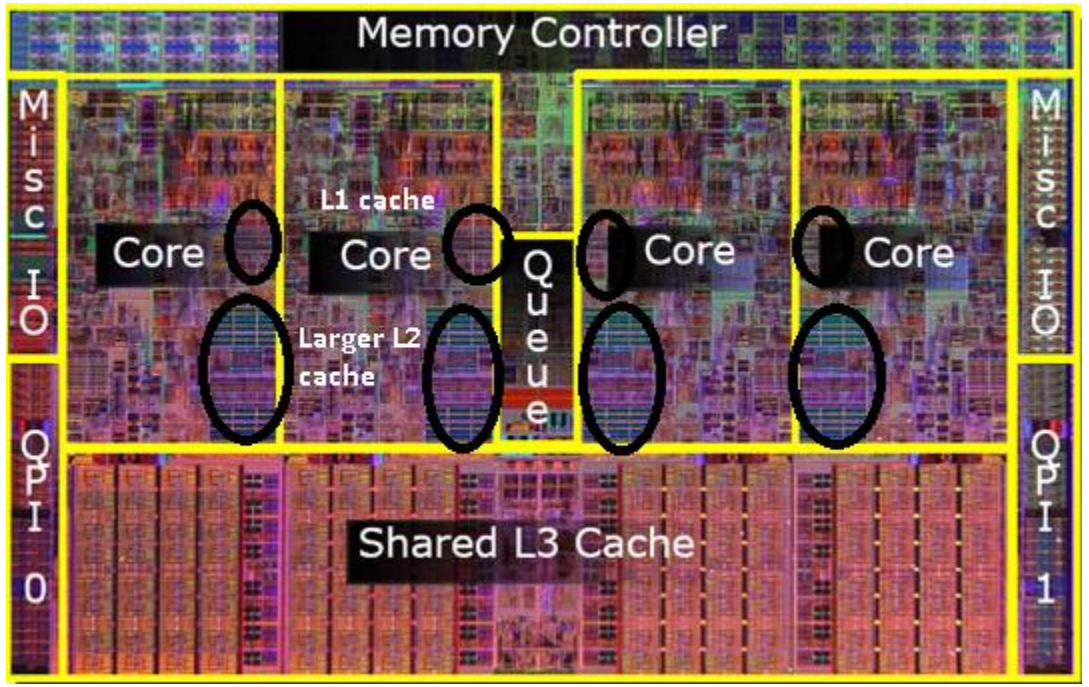
Image courtesy: https://medium.com/

Cache

- Organized into levels on the CPU
  - L1 cache (2 to 64 KB fastest, on core, I and D*)
  - L2 cache (~256 KB medium speed, on core, D only)
  - L3 cache (~ 8MB slowest, across cores, D only)

*I -> instruction, D-> data

- Access latencies
  - L1 cache: 4 cycles (~27x faster than main mem.)
  - L2 cache: 11 cycles (~10x faster than main mem.)
  - L3 cache: 39 cycles (~3x faster than main mem.)
  - Main memory: 107 cycles



- The unit of transfer between cache and main memory is called a **cache line** (typically 64 bytes) and the unit of transfer between internal and external memory is called a **page**

[cpu - Where exactly L1, L2 and L3 Caches located in computer? - Super User]

Locality of Reference

- Operating system designers have developed mechanisms to make most memory accesses to be fast by making use of **locality of reference** properties.
- Two types of locality of reference
  - Spatial locality
  - Temporal locality

- Spatial locality
  - If a program accesses a certain memory location x, then there is increased likelihood that it soon accesses other locations that are near x
  - Example:
    - arr[0] and arr[1] will be next to each other, will be in the same cache line so chances of cache hit are more

```
for(int i=0; i<N;++i) {
   arr[i] = arr[i]*2;
}
```

Locality of Reference

- Temporal locality
  - If a program accesses a certain memory location x, then there is increased likelihood that it accesses that same location x again in the near future.
  - Example:

```
for(int i=0; i<N;++i) {
    arr[i] = arr[i]*2;
}
```

    - The loop variable i will be accessed again in the next iteration to fetch the next array element.

- Typically, 90% of the time of a program is spent in 10% of its code

Dealing with large data

- When the amount of data to process is so large that it cannot fit in the main memory, you have to rely on secondary storage
  - Memory blocks used in transfer between secondary storage and main memory are called **disk blocks** and the transfers are called **disk transfers**.
- Access latencies vary drastically going from the top to bottom levels of memory hierarchy therefore we must minimize the number of disk transfers

Type	M	B	Latency	Bandwidth	$/GB/mo[1]
L1	10K	64B	2ns	80G/s	-
L2	100K	64B	5ns	40G/s	-
L3	1M/core	64B	20ns	20G/s	-
RAM	GBs	64B	100ns	10G/s	1.5
SSD	TBs	4K	0.1ms	5G/s	0.17
HDD	TBs	-	10ms	1G/s	0.04
S3	∞	-	150ms	∞	0.02[2]

[1,2] Memory Hierarchy - Algorithmica

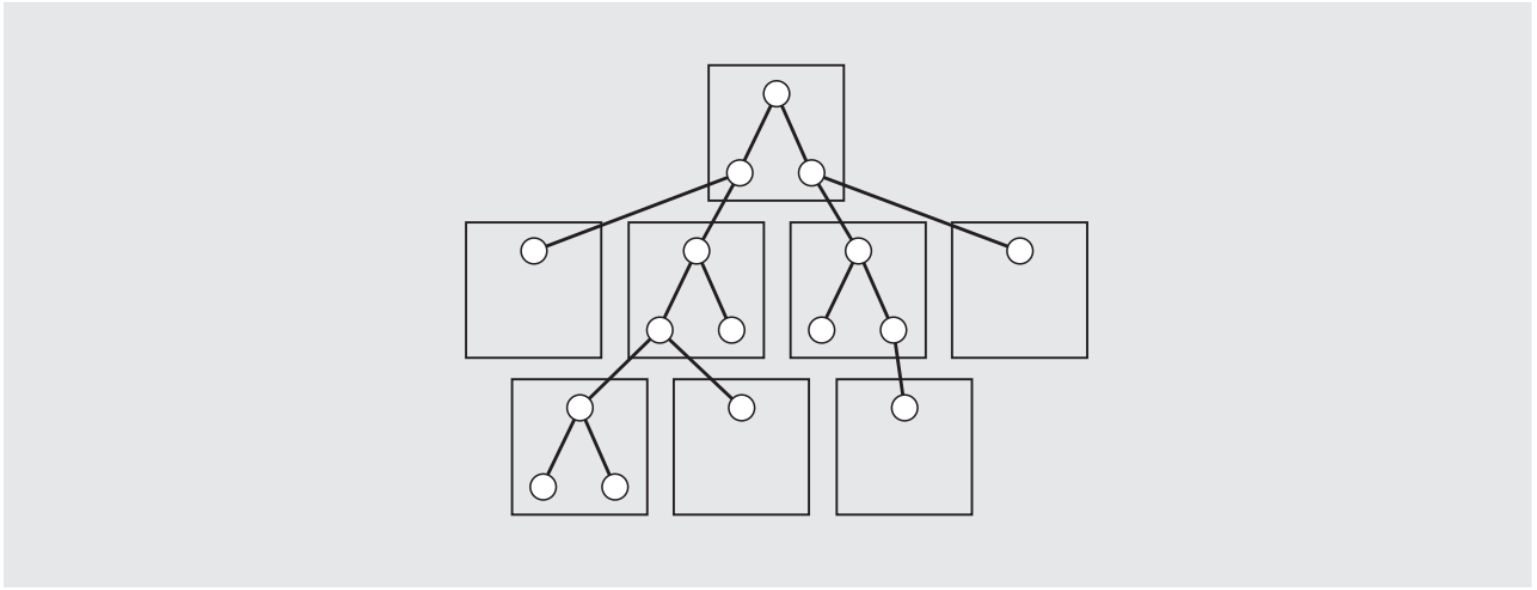
Motivation for B-trees

- One way to minimize the disk transfers is to use an array-based sequence
  - A sequential search of an array can be performed using only O(n/B) block transfers because of spatial locality of reference, where B denotes the number of elements that fit into a block.
  - Due to block transfer, when the first array element is accessed, the first B elements are fetched from the memory into cache
  - This works only in case when we have contiguous memory allocations as in an array

Motivation for B-trees

- We can use a sorted array and use binary search
  - This might improve the performance to O(log(n)) but it might not obtain significant benefit from block transfers because each query during a binary search is likely in a **different block**

FIGURE 7.2 Nodes of a binary tree can be located in different blocks on a disk.



Motivation for B-trees

- Other logarithmic time data structures like AVL, BST or skiplists could be useful for smaller datasets which can fit in internal memory

- We can perform map queries and updates using only $O(\log_B(n)) = O(\log(n)/\log(B))$ transfers

What are Multiway Search Trees?

- Special type of a tree where internal nodes can have more than 2 children.
- Map items stored in a search tree are pairs of the form (k,v), where k is the key and v is the value associated with the key.
- In the following definition:
  - w is a node of an ordered tree,
  - w is a d-node if it has d children

What are Multiway Search Trees?

- Multiway Search Tree is an ordered tree T that has the following properties:
  - Each internal node of T has at least two children.
    - Each internal node is a d-node such that $d \geq 2$.
  - Each internal d-node w of T with children $c_1,\ldots,c_d$ stores an ordered set of $d-1$ key-value pairs $(k_1,v_1),\ldots,(k_{d-1},v_{d-1})$, where $k_1 \leq \cdots \leq k_{d-1}$.
  - Let us conventionally define $k_0 = -\infty$ and $k_d = +\infty$. For each item (k,v) stored at a node in the subtree of w rooted at c_i, $i = 1,\ldots,d$, we have that $k_{i-1} \leq k \leq k_i$.

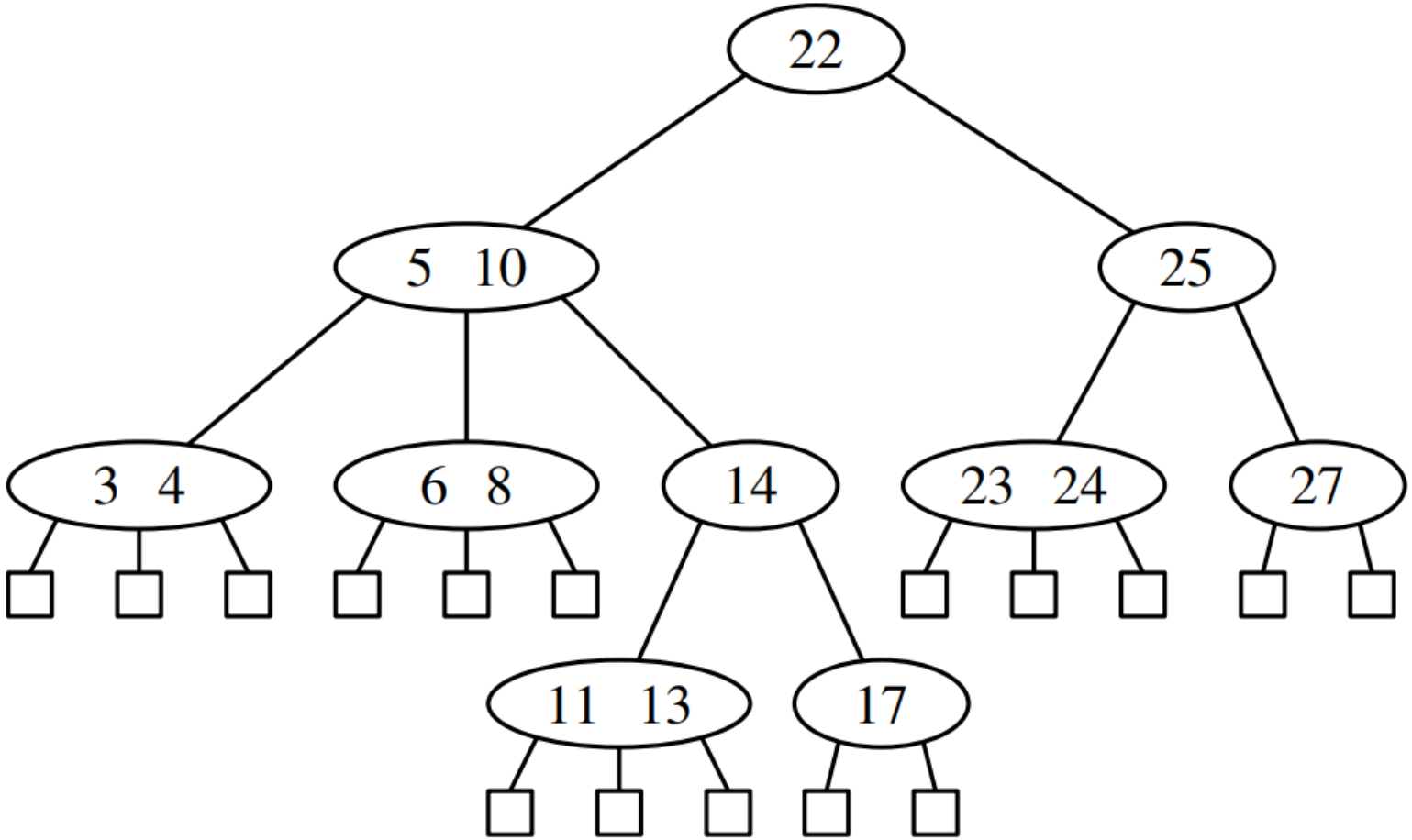
- External nodes of a multiway search do not store any data and serve only as "placeholders."

Multiway Search Tree: Definition

- A multiway search tree of order m, or an m-way search tree, is a multiway tree in which

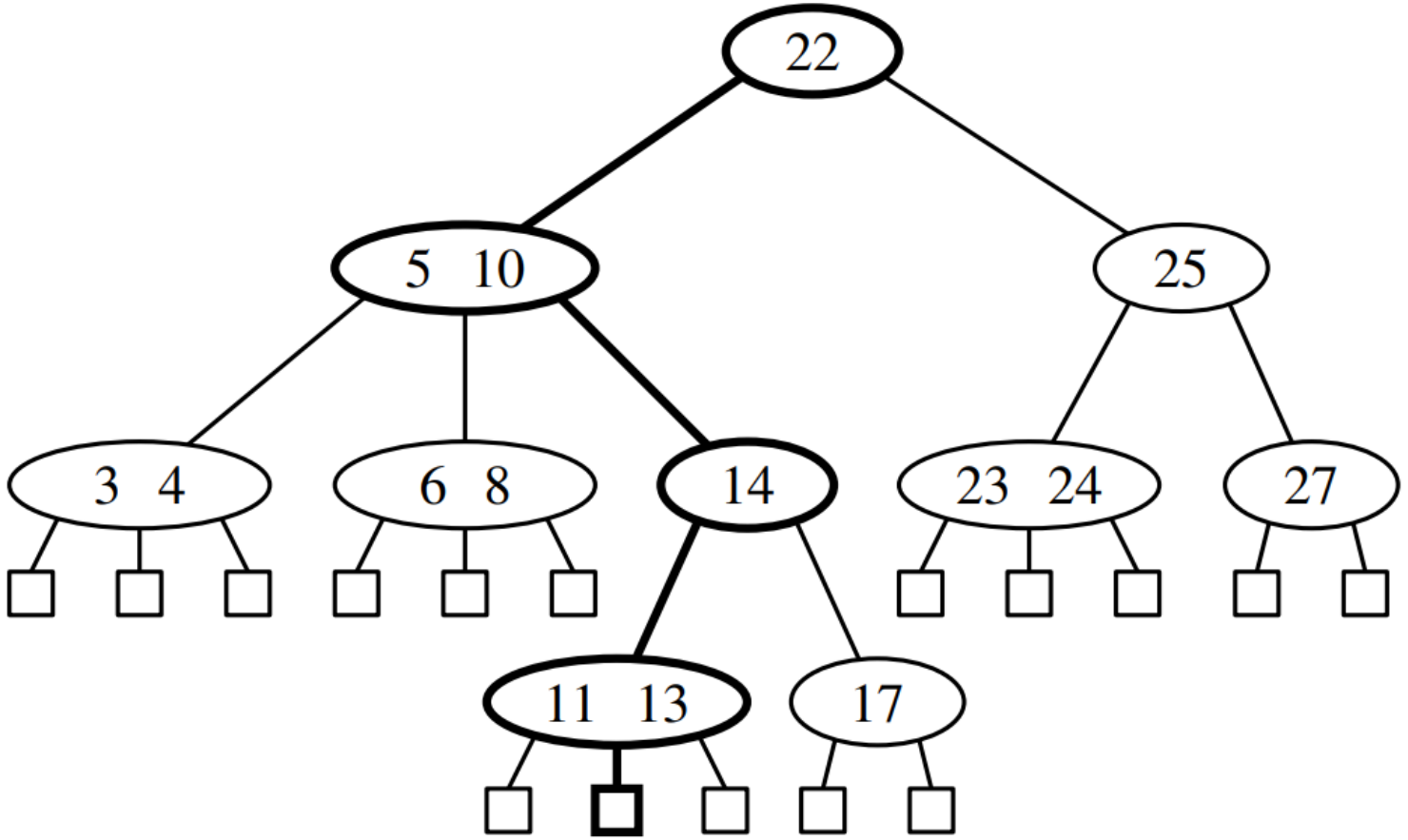
1. Each node has m children and m – 1 keys.
2. The keys in each node are in ascending order.
3. The keys in the first i children are smaller than the i^{th} key.
4. The keys in the last m – i children are larger than the i^{th} key.

Example



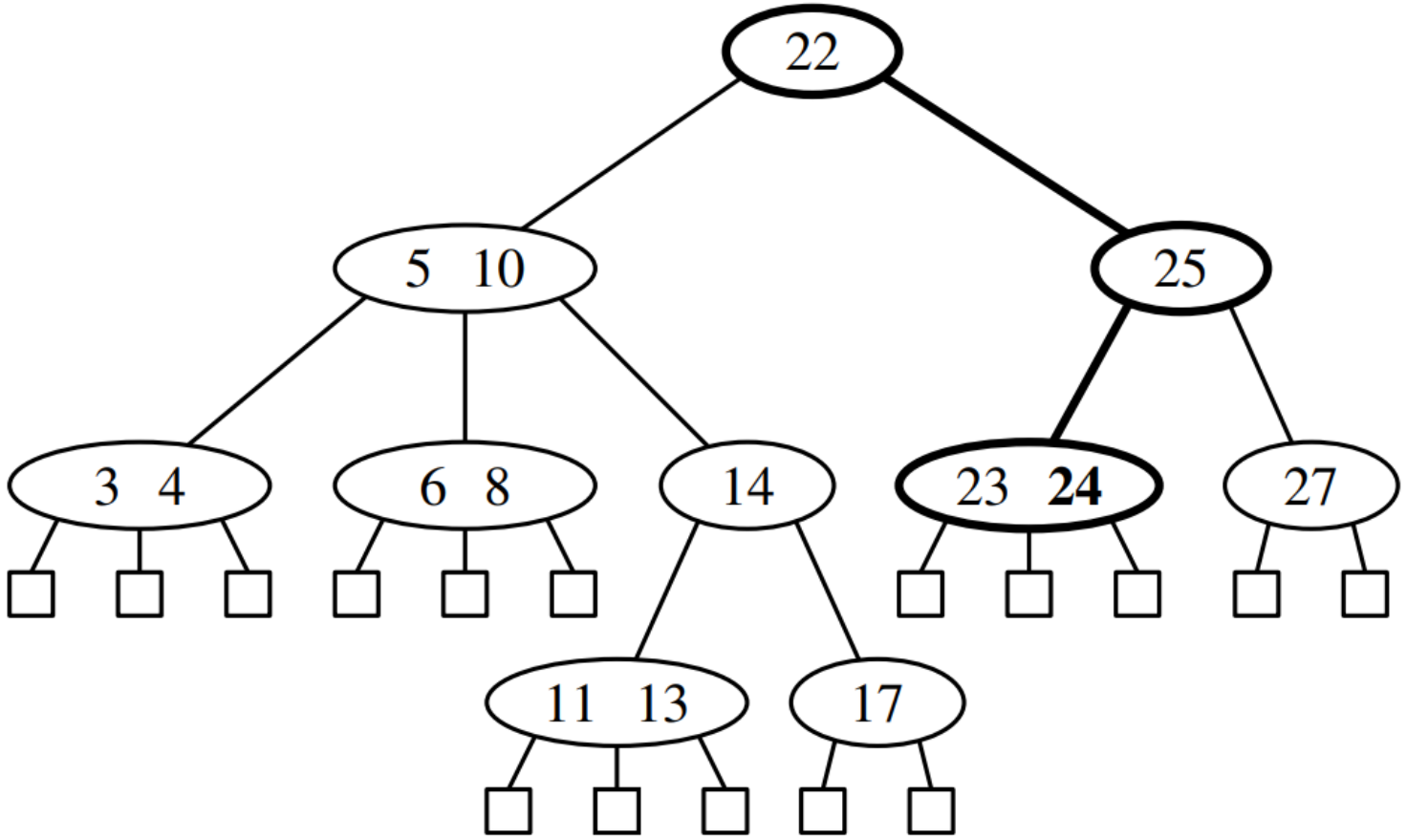
Operations on Multiway Search Trees

- Example of unsuccessful search path: search(12)



Operations on Multiway Search Trees

- Example of successful search path: search(24)



What is a B-tree?

- AVL, BST assume the entire tree must be in primary memory
  - So these trees are useful for small datasets
- B-tree are a special kind of self balancing BST
- A B-tree is a balanced multiway tree (also known as the balanced sort tree) in which nodes are arranged on the basis of in-order traversal.
- Key idea of B-tree
  - **reduce** the number of **disk accesses** by storing **more than one keys per node** in **sorted order** so that **cache hits** may be maximized.

B-tree: Definition

- A B-tree of order m is a multiway search tree with the following properties:

1. The root has at least two subtrees unless it is a leaf.
2. Each nonroot and each nonleaf node holds k – 1 keys and k pointers to subtrees where $\left\lceil \frac{m}{2} \right\rceil \leq k \leq m$.
3. Each leaf node holds k – 1 keys where $\left\lceil \frac{m}{2} \right\rceil \leq k \leq m$.
4. All leaves are on the same level.

- According to these conditions, a B-tree is always at least half full, has few levels, and is perfectly balanced.

B-tree

- There are certain conditions that must be true for a B-tree:
  - The height of the tree must remain as minimum as possible.
  - Above the leaves of the tree, there should not be any empty subtrees.
  - The leaves of the tree must occur at the same level.
  - All nodes must have some minimum number of children except leaf nodes.

- n is the maximum number of children, and k is the number of keys that each node can contain.
  - There can be at maximum n – 1 key for all nodes.
  - There are at least two children for the root.
  - There are at least n/2 children and at most n non-empty children.
  - The tree gets divided so that values in the left subtree shall be less than the value in the right subtree.

Why use B-Tree?

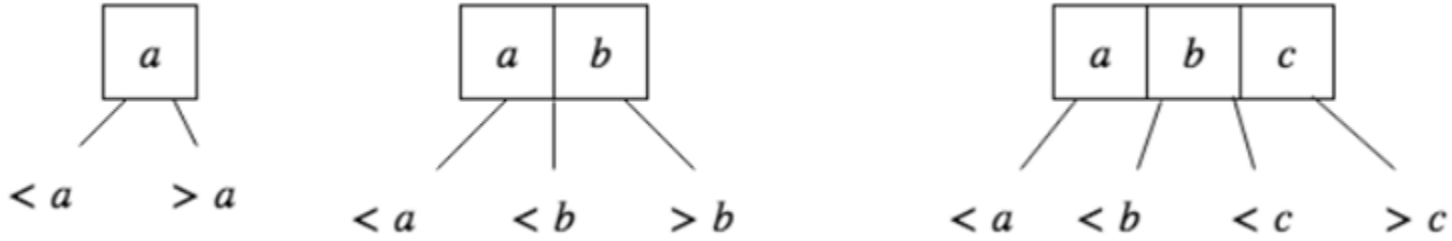
- Reduces the number of reads made on the disk
- Can be easily optimized to adjust its size (that is the number of child nodes) according to the disk size
- Specially designed for handling of bulky amount of data.
- Applicable for developing algorithms for databases and file systems.
- A good choice to opt when it comes to reading and writing large blocks of data

B-Tree: Node definition


```
template class BTreeNode
{
   public:
      BTreeNode();
      BTreeNode(const T&);
   private:
      bool leaf;
      int keyTally;
      T keys[M-1];
      BTreeNode *pointers[M];
      friend BTree;
};
```

2-4 or 2-3-4 Trees

- 2-4 tree (or 2-3-4 tree) is a B-Tree of order 4.
- A node cannot hold more than three keys.
- If a node is full before insertion, we split the node so that the new node can be inserted.
- This type of tree can contain nodes of three types.

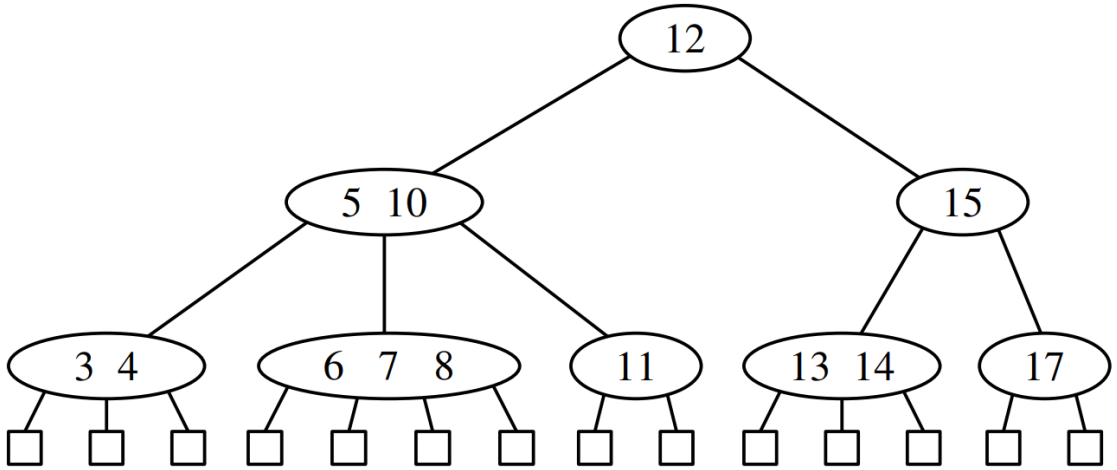


- Why do we have three types of nodes
  - It helps make the tree perfectly balanced by keeping all the leaves at the same level after each insertion and deletion operation.

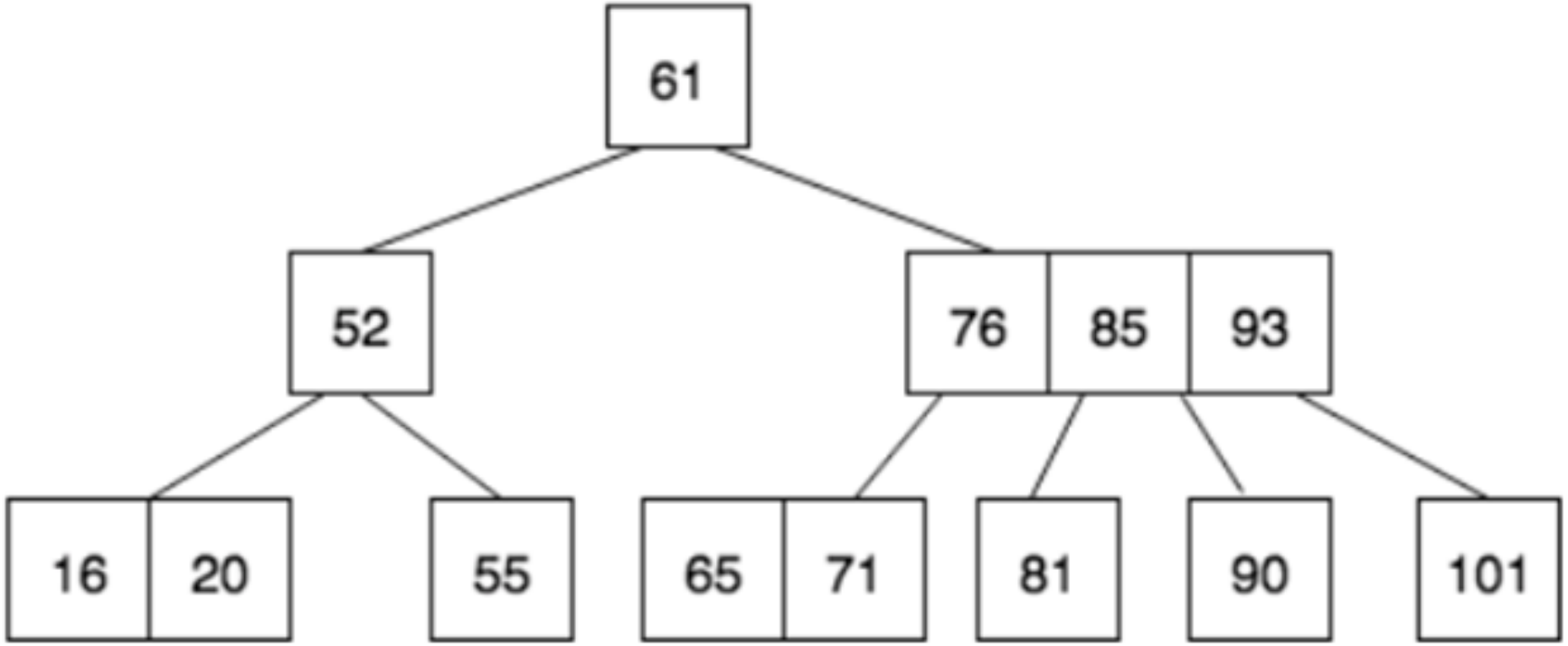
2-3-4 Trees | Algorithm Tutor

2-4 Tree: Definition

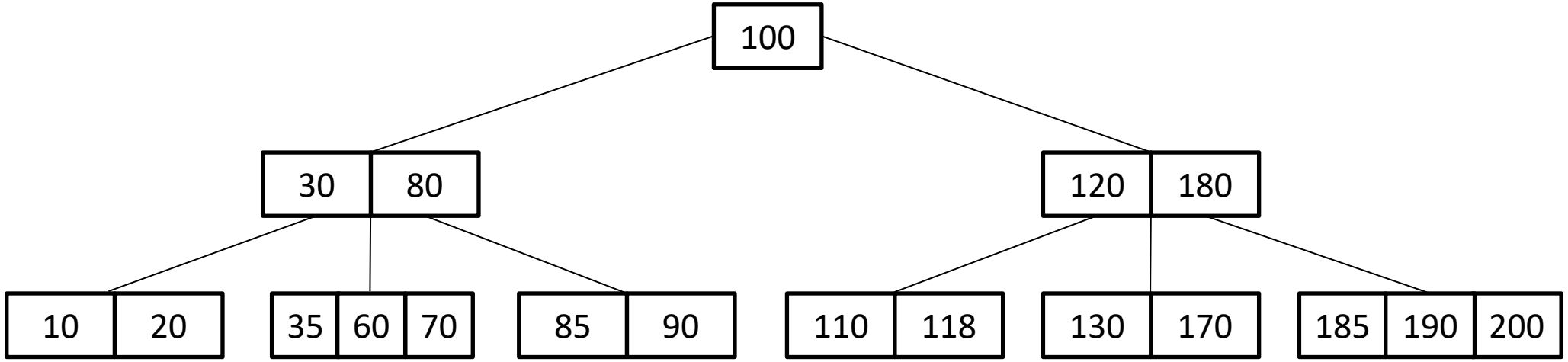
- A multiway search tree that keeps the secondary data structures stored at each node small and also keeps the primary multiway tree balanced
- It achieves these goals by maintaining two simple properties
  - **Size Property:** Every internal node has at most four children.
  - **Depth Property:** All the external nodes have the same depth.



Example

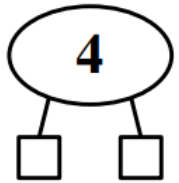


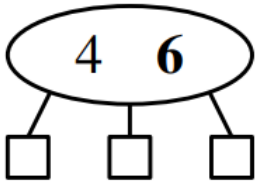
Another Example

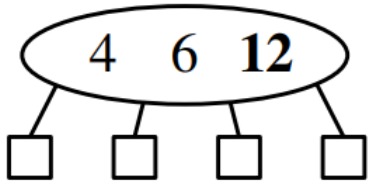


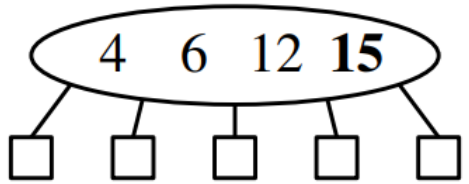
2-4 Tree: Operations - Insert

- Given a set of values: {4, 6, 12, 15, 3, 5, 10, 8}





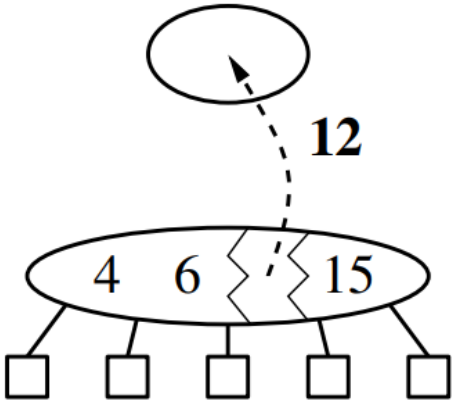


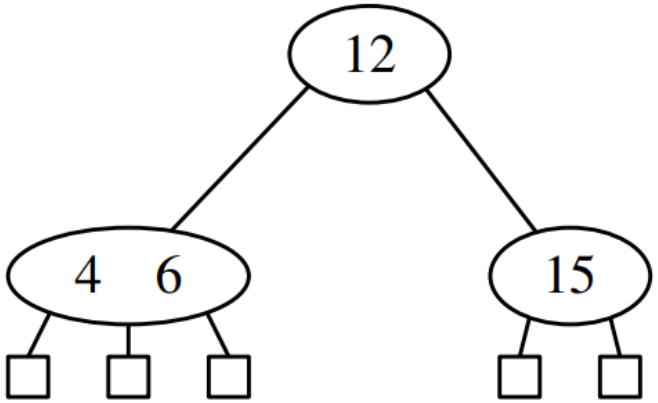


Overflow: split the node

2-4 Tree: Operations - Insert

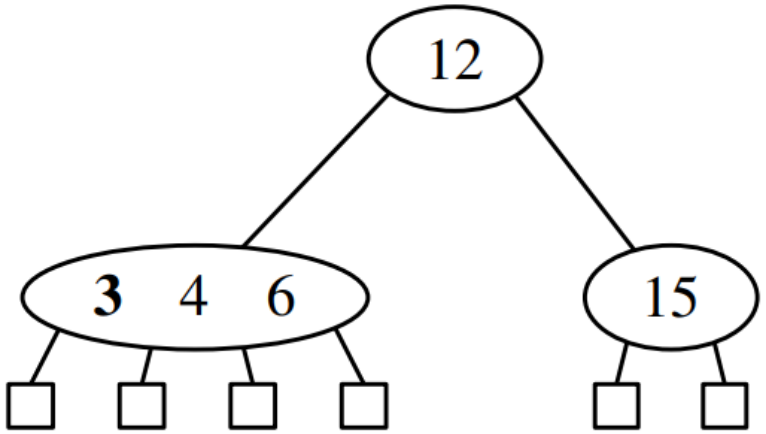
- Given a set of values: {4, 6, 12, 15, 3, 5, 10, 8}

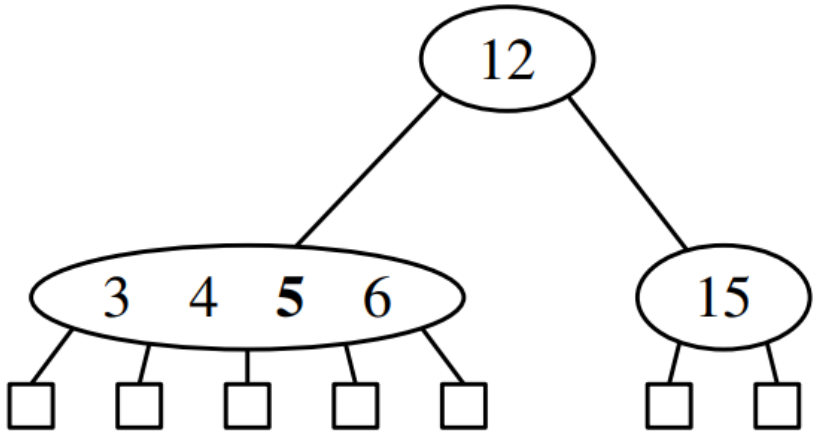




2-4 Tree: Operations - Insert

- Given a set of values: {4, 6, 12, 15, 3, 5, 10, 8}

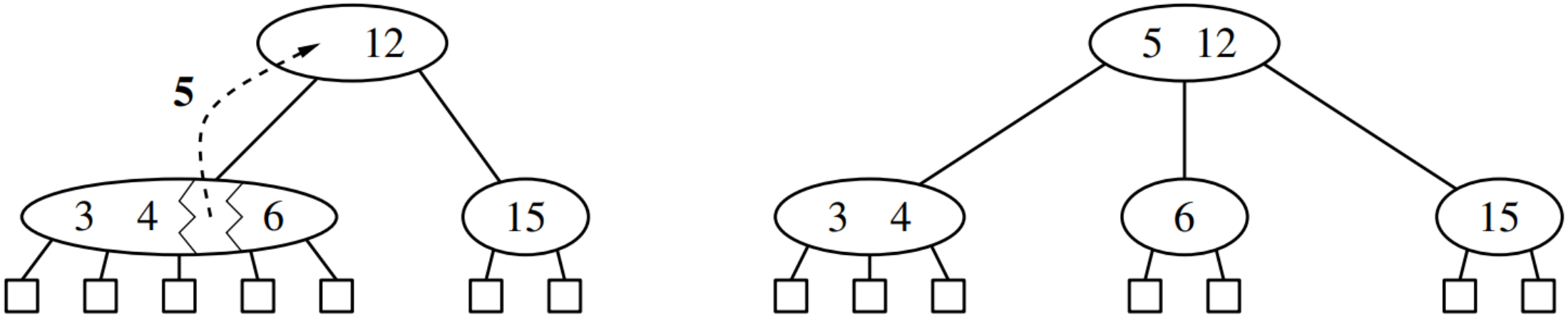




Overflow: split the node

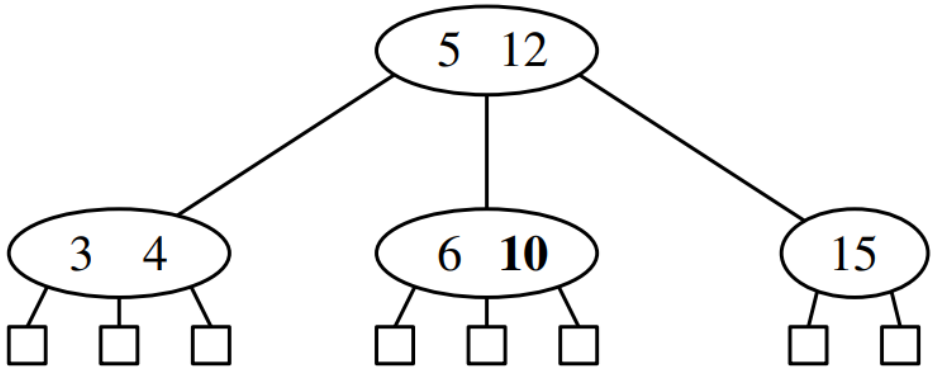
2-4 Tree: Operations - Insert

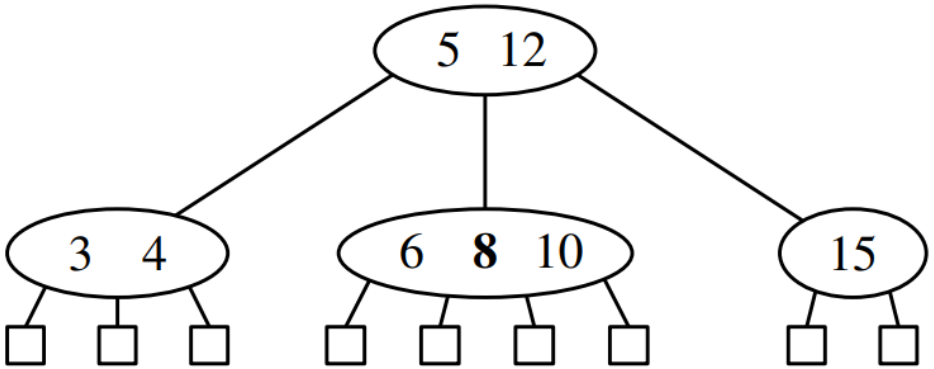
- Given a set of values: {4, 6, 12, 15, 3, 5, 10, 8}



2-4 Tree: Operations - Insert

- Given a set of values: {4, 6, 12, 15, 3, 5, 10, 8}





2-4 Tree: Inserting a new node

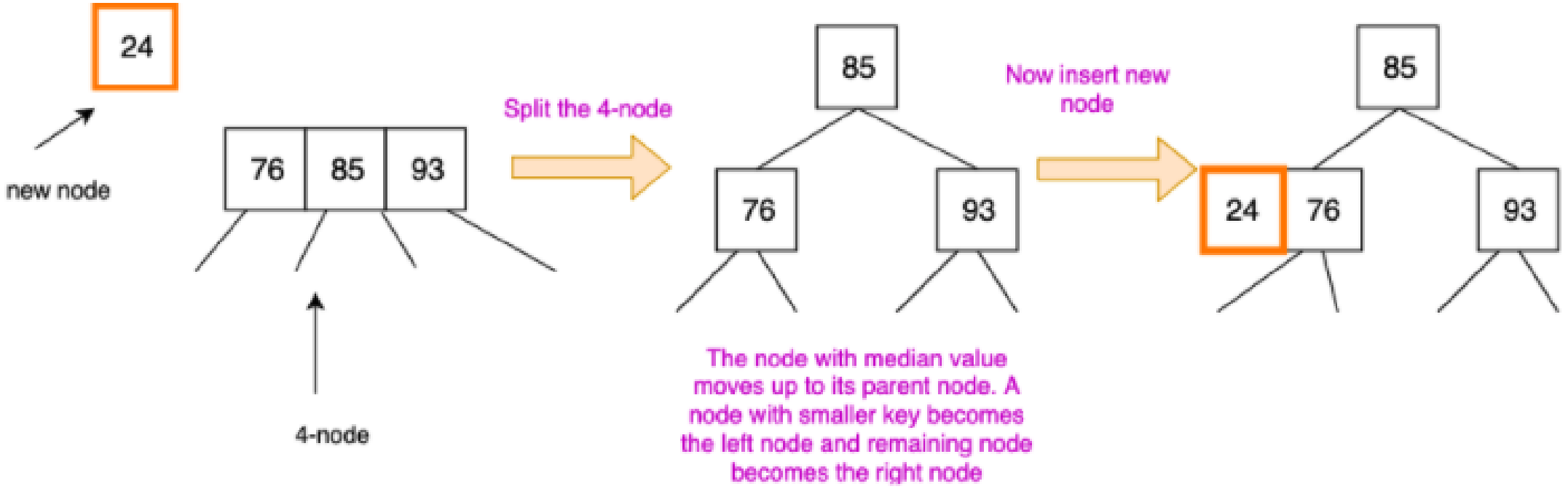
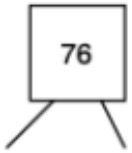


Figure 3: Splitting of a 4-node before inserting a new node.

[2-3-4 Trees | Algorithm Tutor]

Insert 76

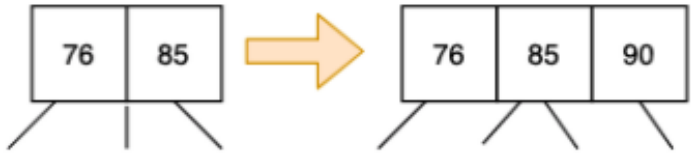


Since the tree is empty, the new node becomes the root of the tree

Insert 85

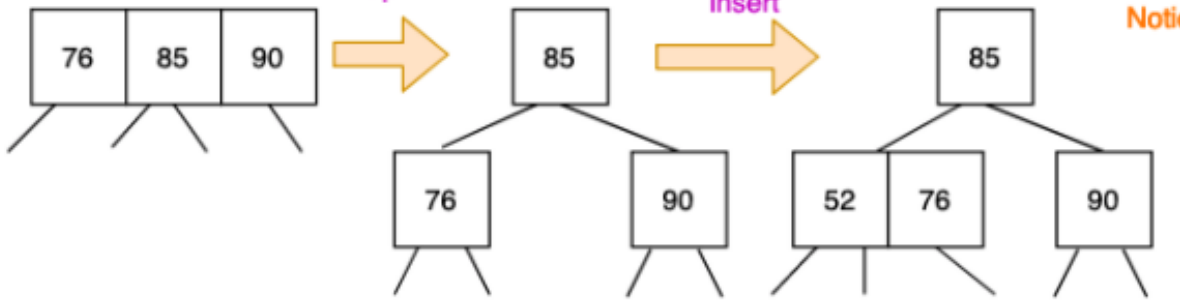
2-node becomes 3-node

Insert 90



3-node becomes 4-node

Insert 52

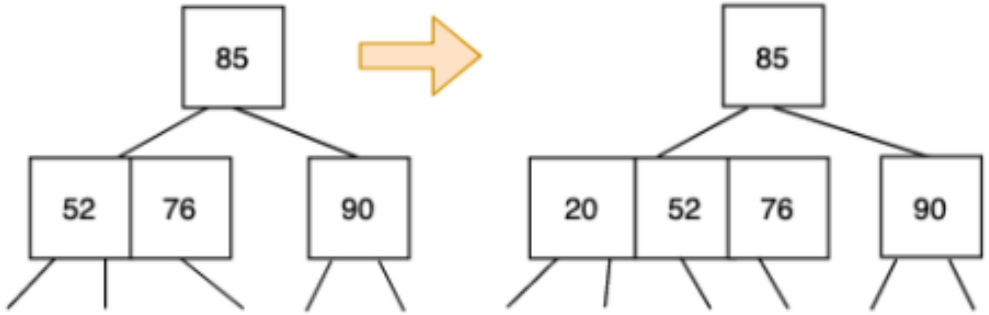


Notice the height change

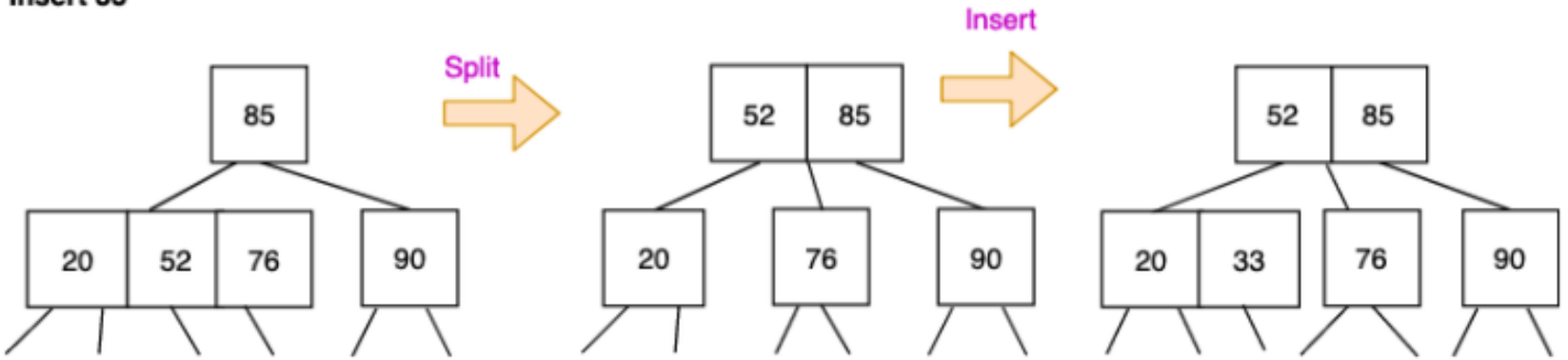
[2-3-4 Trees | Algorithm Tutor]

Sequence of Inserts

Insert 20



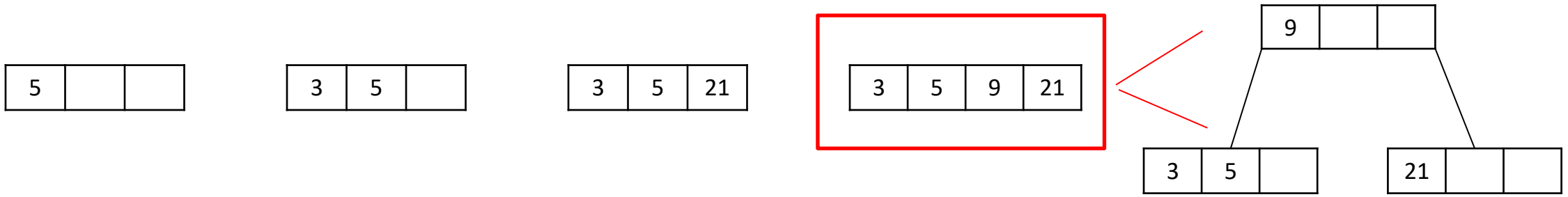
Insert 33



[2-3-4 Trees | Algorithm Tutor]

Insertion - Exercise

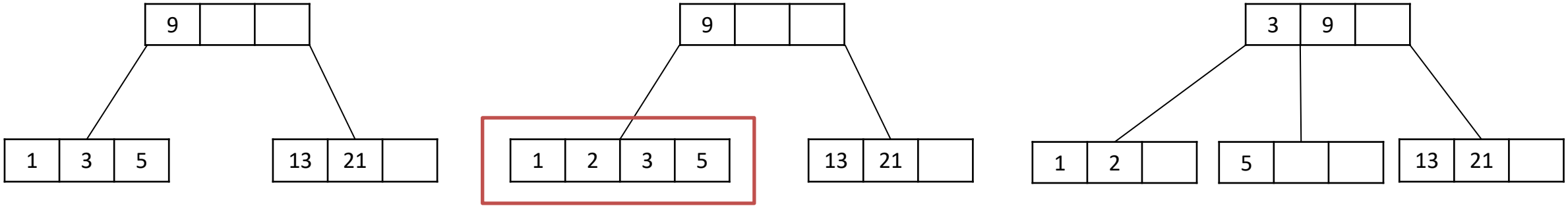
- Construct a 2-4 tree with the following values:
  - 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8



Overflow: split the node

Insertion - Exercise

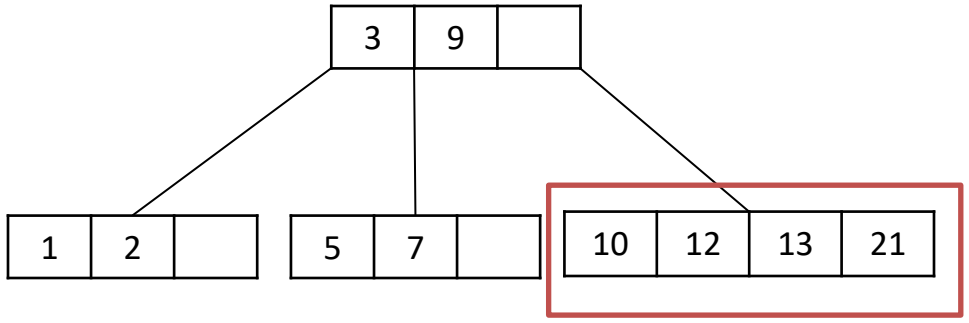
- Construct a 2-4 tree with the following values:
  - 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8



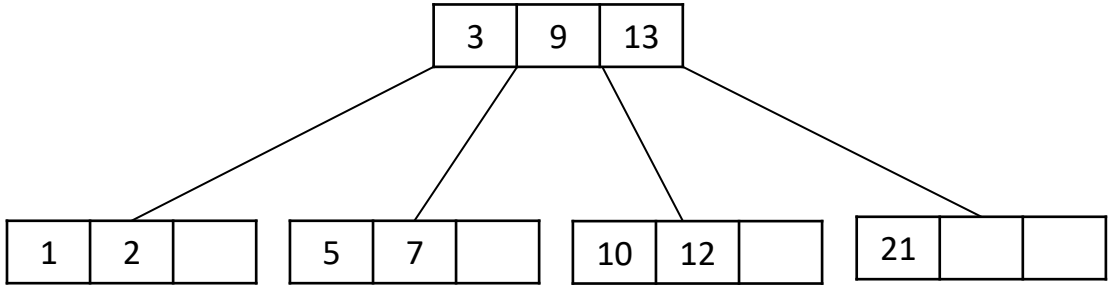
Overflow: split the node

Insertion - Exercise

- Construct a 2-4 tree with the following values:
  - 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8

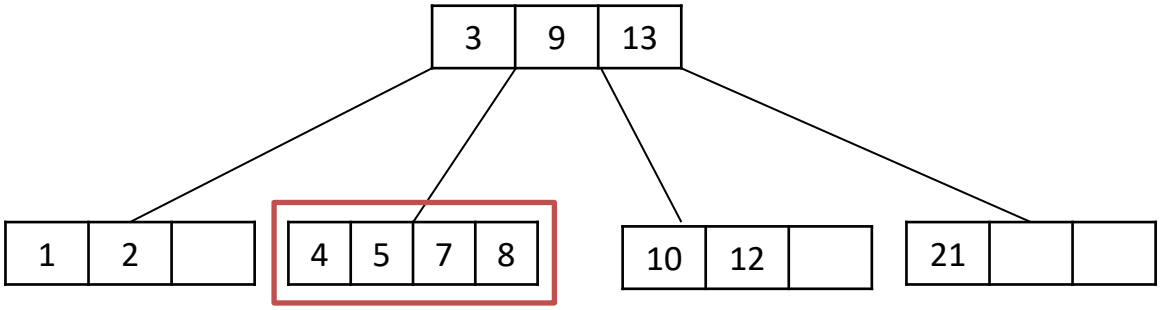


Overflow: split the node

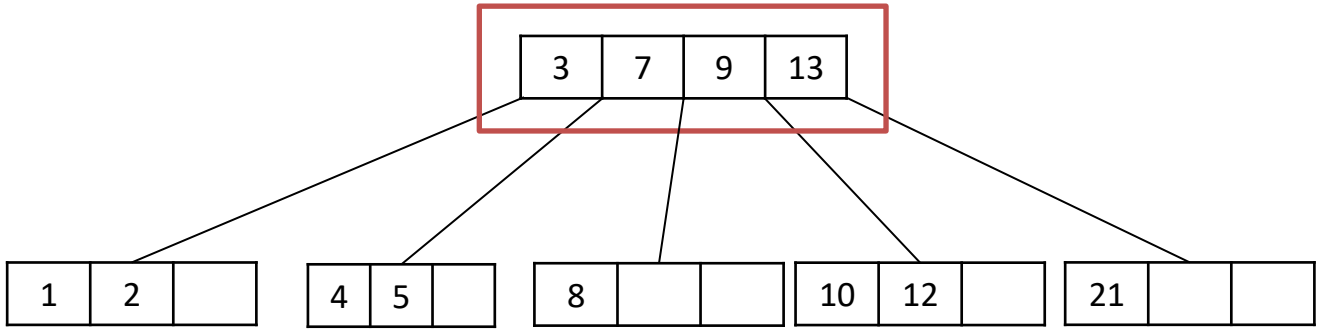


Insertion - Exercise

- Construct a 2-4 tree with the following values:
  - 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8



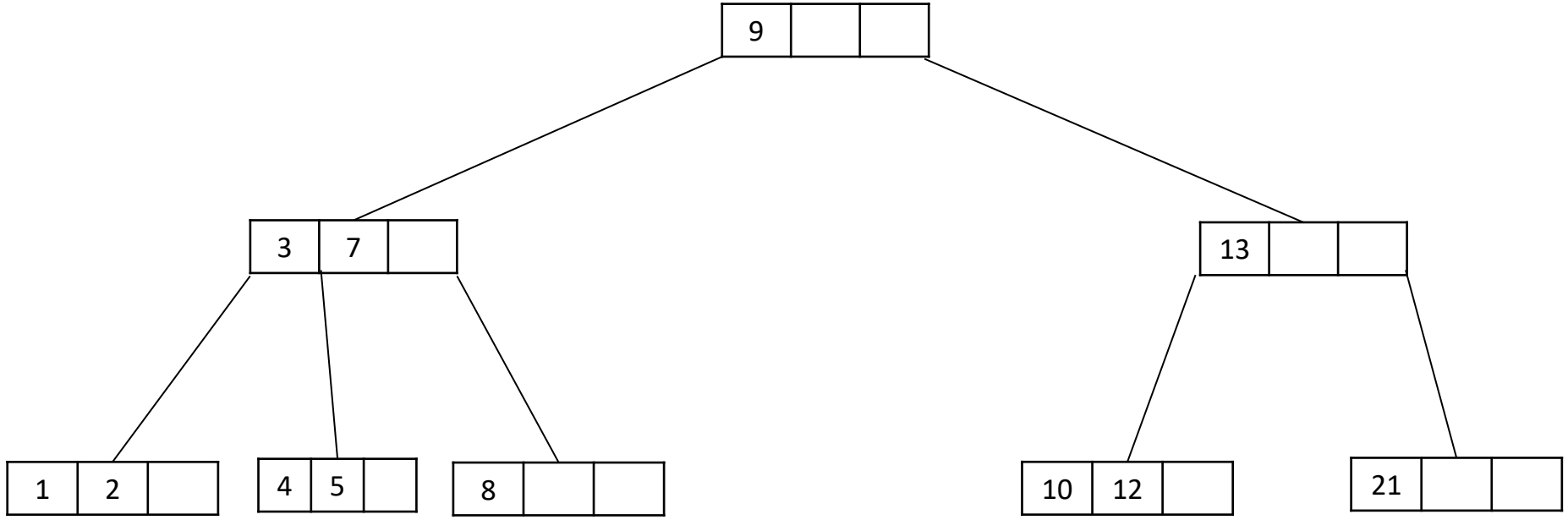
Overflow: split the node



Overflow: split the node

Insertion - Exercise

- Construct a 2-4 tree with the following values:
  - 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8



Insertion - Exercise

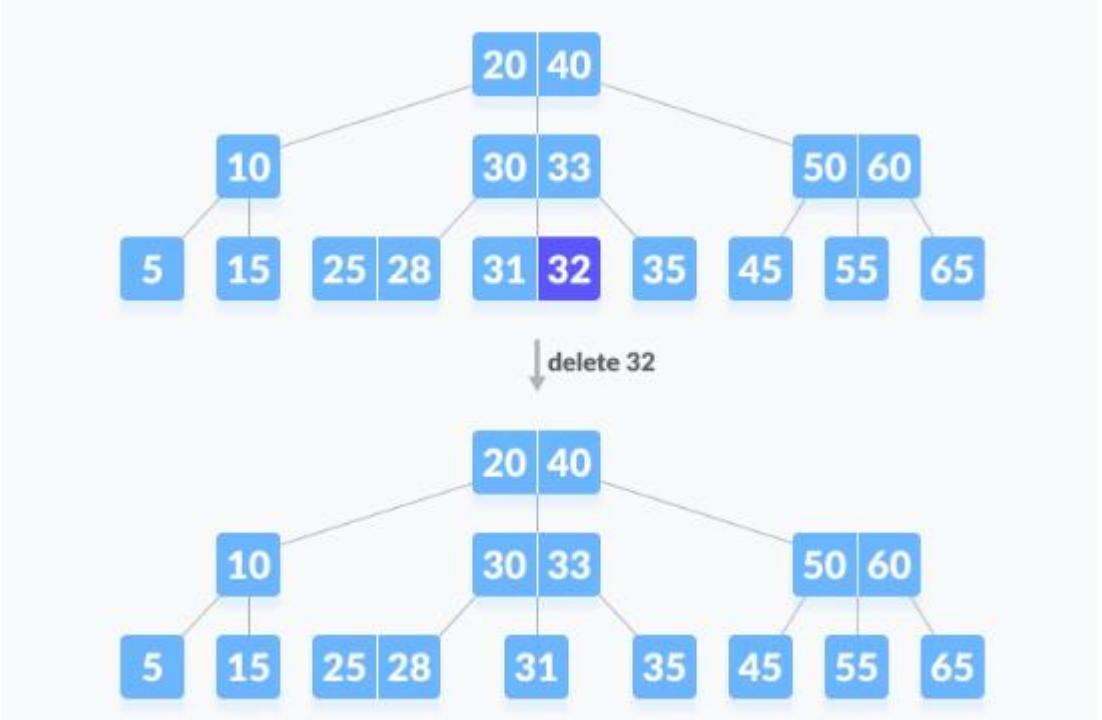
- Insert the following values in a 2-4 tree.
  - 21,14,39,64,11,78,51,37,20,7,19

Deleting a node from B-Tree

- leaf node
  - with more than min keys:
    - Delete the key, rest is good.
  - with min key:
    - Try to borrow a key from left or right sibling if they have more than mid child. This would happen via rotation.
  - Else
    - Merge with sibling using parent
    - If parent is also min code, apply merging at parent's level too.
- Internal node
  - if the node has more than min child
    - Delete the key, rest is good.
  - Replace by inorder predecessor/inorder successor if left/right child has more than min child
  - Else merge left and right child
  - If the node to be deleted has less then min nodes (after deletion), then merge this node with its siblings.

Deleting a leaf node – Case I

- The deletion of the key does not violate the property of the minimum number of keys a node should hold.



https://www.programiz.com/dsa/deletion-from-a-b-tree

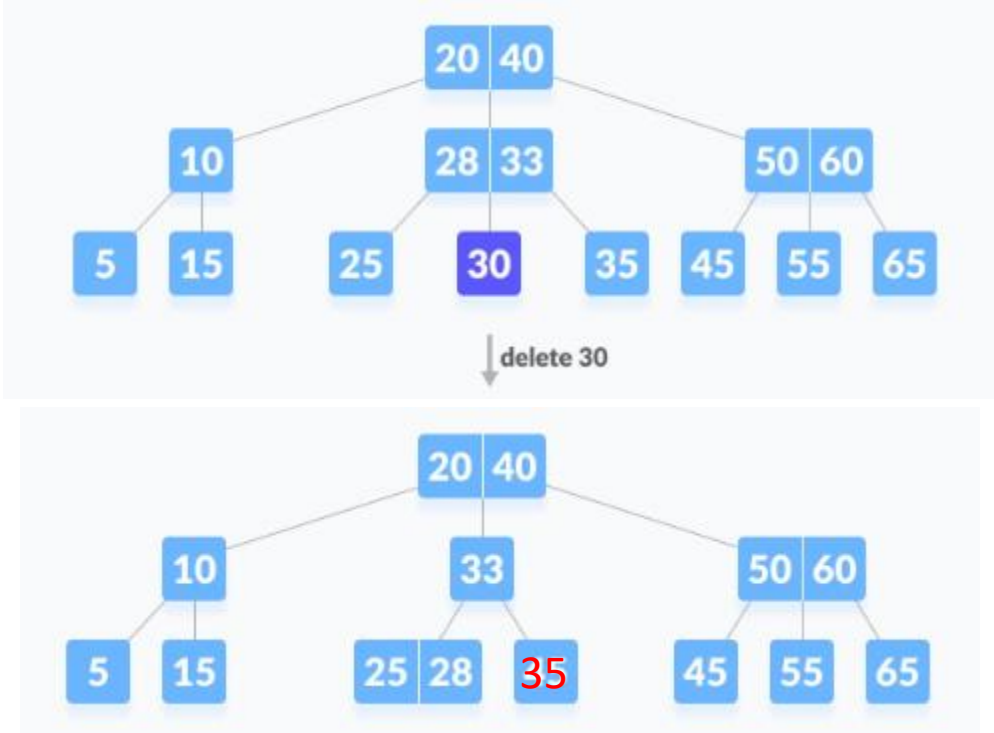
Deleting a leaf node – Case II

- The deletion of the key violates the property of the minimum number of keys a node should hold. In this case, we borrow a key from its immediate neighbouring sibling node in the order of left to right.

https://www.programiz.com/dsa/deletion-from-a-b-tree

Deleting a leaf node – Case III

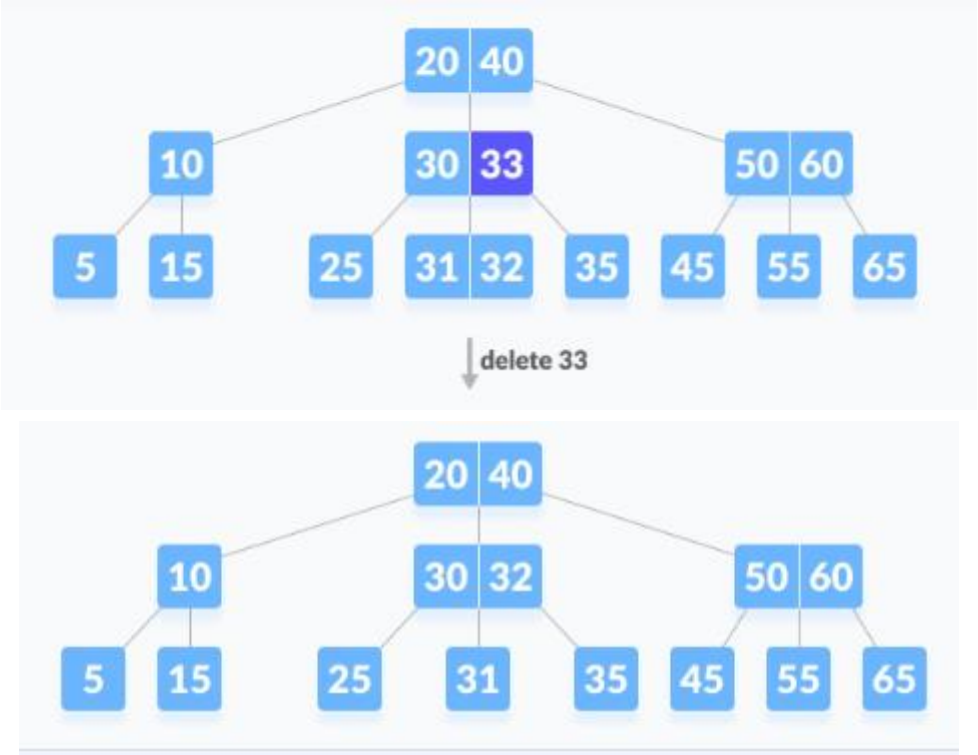
- If both the immediate sibling nodes already have a minimum number of keys, then merge the node with either the left sibling node or the right sibling node. **This merging is done through the parent node.**



https://www.programiz.com/dsa/deletion-from-a-b-tree

Deleting an internal node – Case I

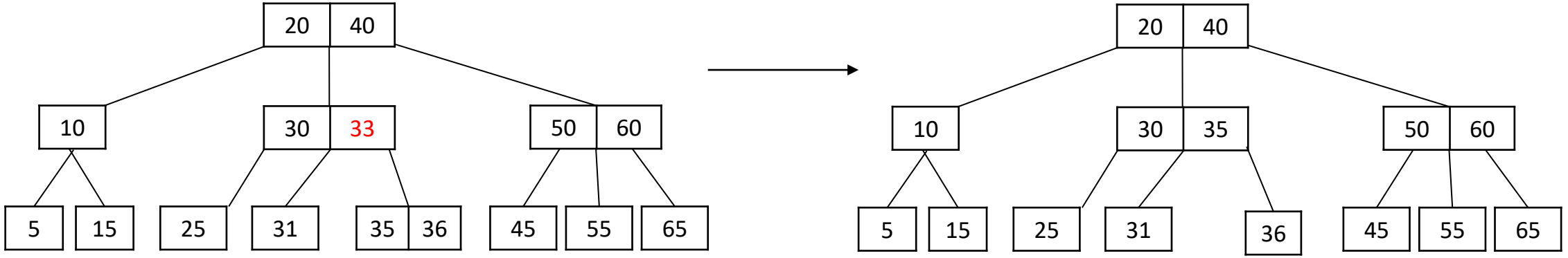
- The internal node, which is deleted, is replaced by an inorder predecessor if the left child has more than the minimum number of keys.



https://www.programiz.com/dsa/deletion-from-a-b-tree

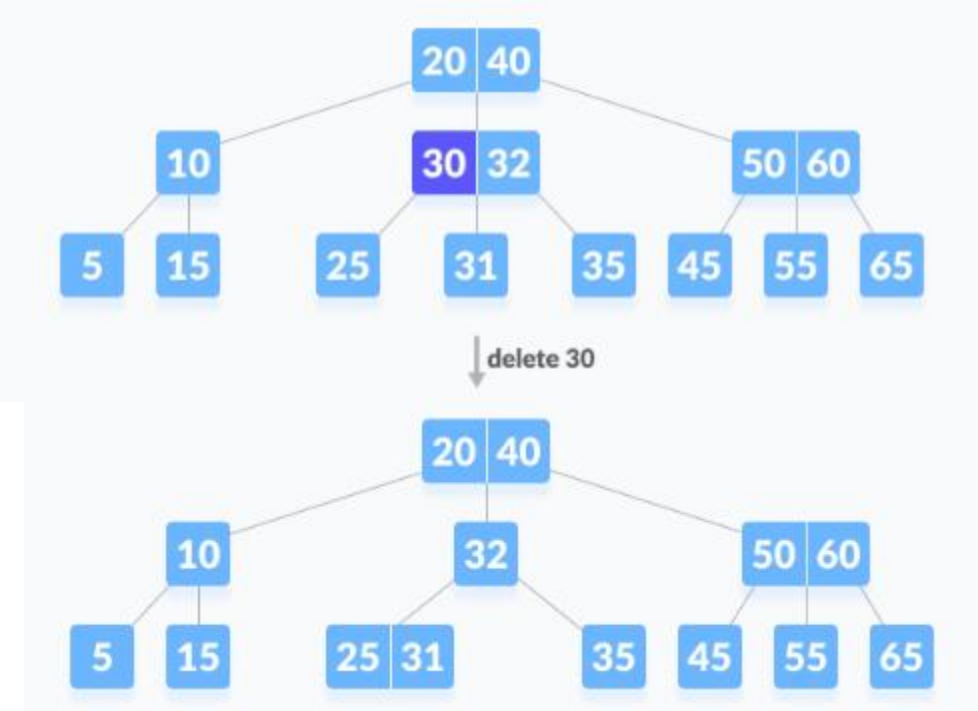
Deleting an internal node – Case II

- The internal node, which is deleted, is replaced by an inorder successor if the right child has more than the minimum number of keys.



Deleting an internal code – Case III

- If either child has exactly a minimum number of keys then, merge the left and the right children.



https://www.programiz.com/dsa/deletion-from-a-b-tree

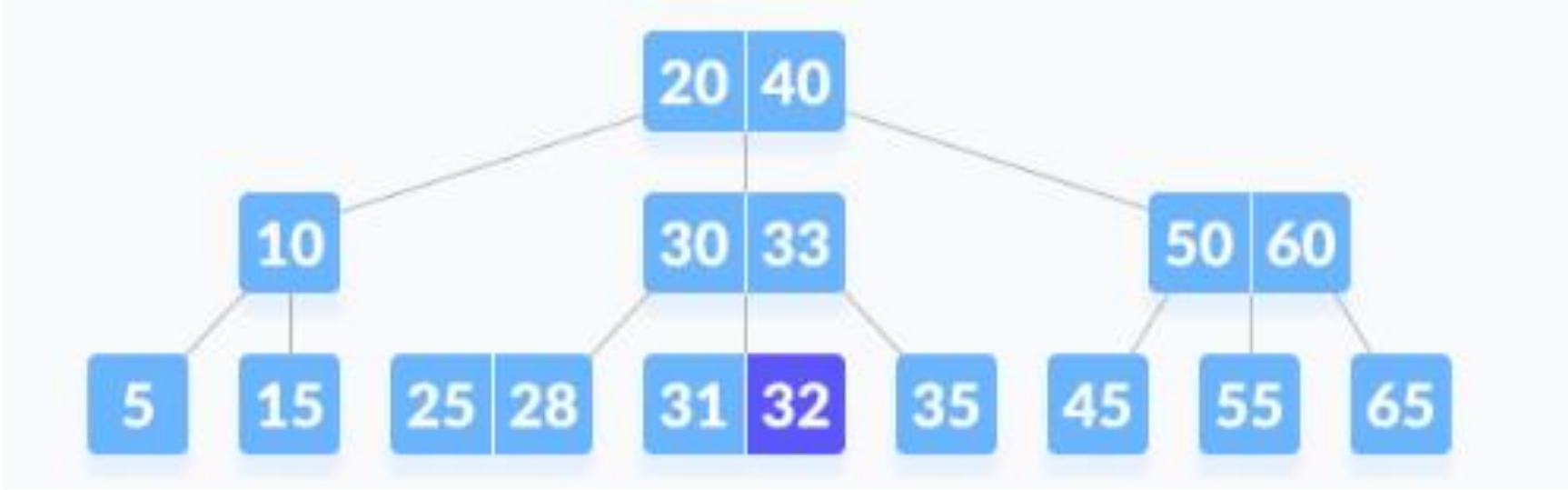
Deleting an internal node – Case IV

- In this case, the height of the tree shrinks. If the sibling also has only a minimum number of keys then, merge the node with the sibling along with the parent. Arrange the children accordingly (increasing order).

https://www.programiz.com/dsa/deletion-from-a-b-tree

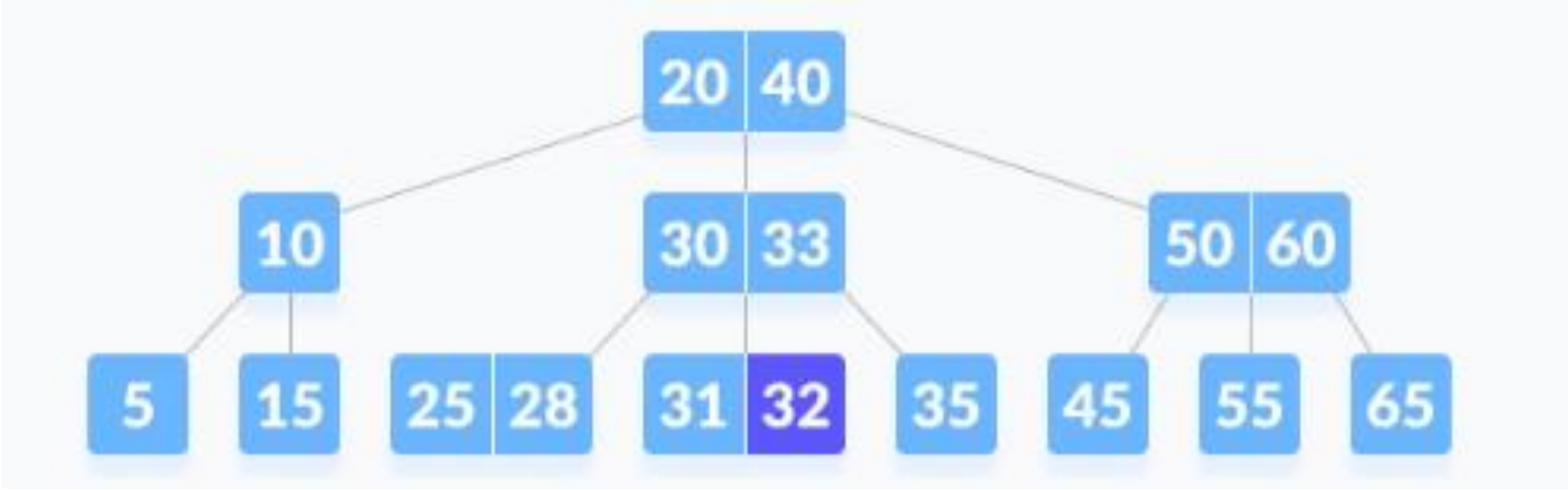
Exercise

- Delete 35.



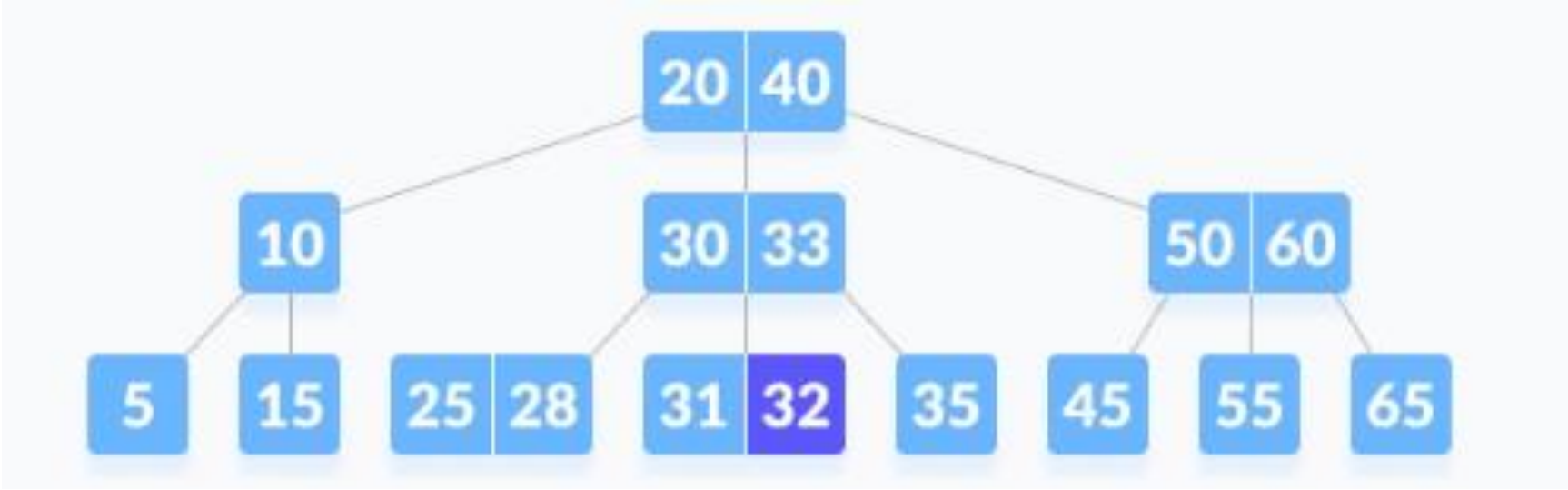
Exercise

- Delete 45.



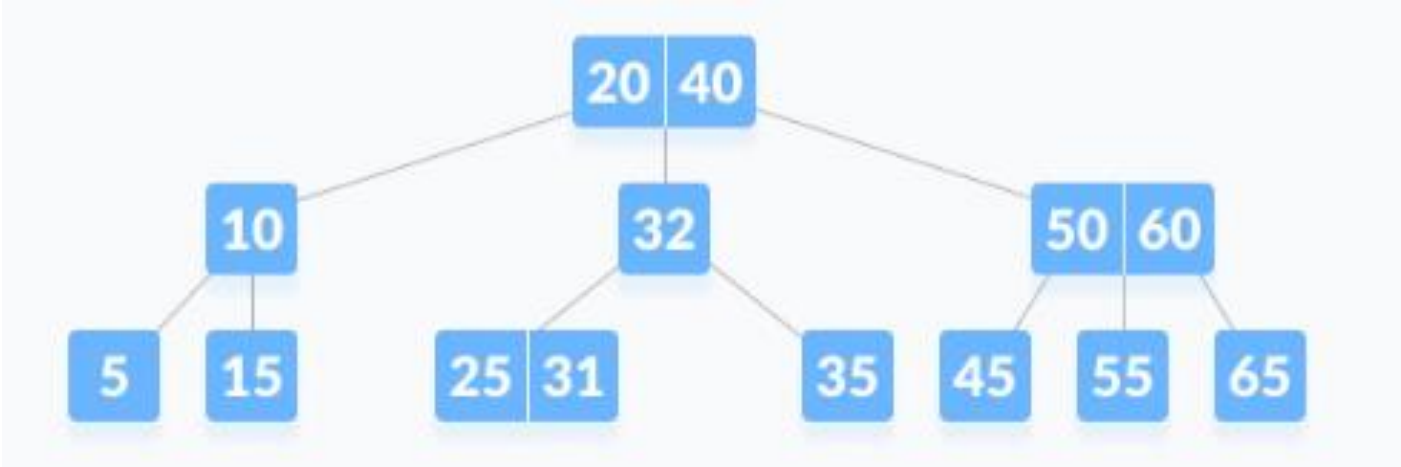
Exercise

- Delete 30.



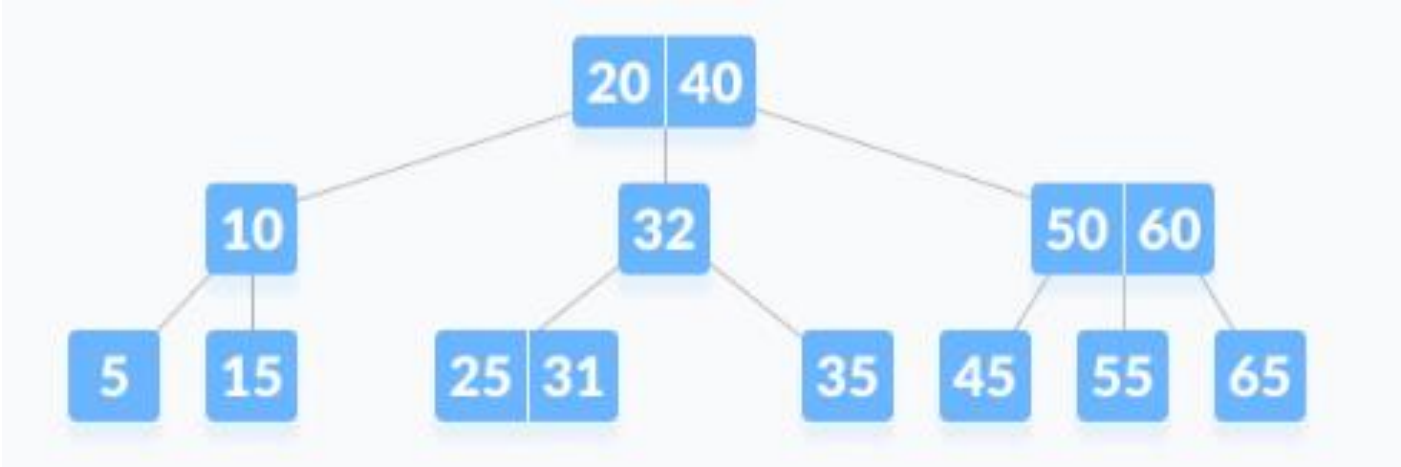
Exercise

- Delete 60.



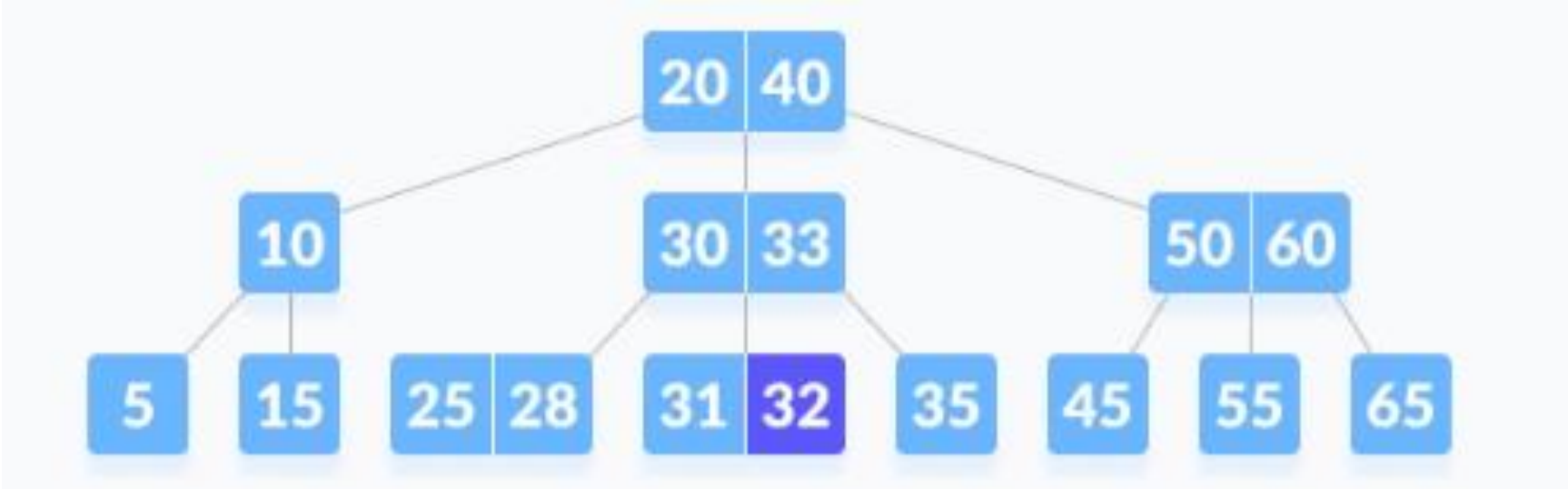
Exercise

- Delete 10.



Exercise

- Delete 5.



Max number of keys in B-Tree

- Q) What is the maximum number of keys in a B-Tree of order m of height h?
- Ans. A B-tree of order m of height h will have the maximum number of keys when all nodes are completely filled.
- So, the B-tree will have $n = (m^{h+1} – 1)$ keys in this situation.

Complexity of find, insertion and deletion

- Find
  - By the search property of 2-4 Trees, we have at most 4 nodes at depth 1, 4^2 at depth 2, and 4^h nodes at depth h.
  - By the depth property of 2-4 Trees, all external nodes have the same depth. This implies that we have at least 2 nodes at depth 1, 2^2 nodes at depth 2 and 2^h nodes at depth h.
  - Since an n-way multiway tree has n+1 external nodes, we have,

$$2^h \leq n + 1 \leq 4^h$$

  - Taking $\log_2$ on both sides

$$h \leq \log_2(n+1) \leq 2h$$

  - This implies that the height of 2-4 is O(log(n))
  - Since a 2-4 Tree is balanced and complete, the find function will take O(log(n)) time

Complexity of find, insertion and deletion

- Insert
  - The maximum no. of children of any node in a 2-4 tree (dmax) is at most 4
  - The original search for the placement of new key k uses O(1) time at each level
  - This implies O(log(n)) time overall, since the height of the tree is O(log(n)).
  - The modifications to a single node to insert a new key and child can be implemented to run in O(1) time, as can a single split operation.
  - The number of cascading split operations is bounded by the height of the tree, and so that phase of the insertion process also runs in O(log(n)) time.
  - Therefore, the total time to perform an insertion in a (2,4) tree is O(log(n))

Complexity of find, insertion and deletion

- Delete
  - Removing an item from a 2-4 tree preserves the depth property but it might violate the size property (underflow condition)
  - To remedy an underflow, we check whether an immediate sibling of w is a 3-node or a 4-node.
    - If we find such a sibling s, then we perform a transfer operation, in which we move a child of s to w, a key of s to the parent u of w and s, and a key of u to w.
    - If w has only one sibling, or if both immediate siblings of w are 2-nodes, then we perform a **fusion** operation, in which we merge w with a sibling, creating a new node w , and move a key from the parent u of w to w
  - In the worst case, the underflow propagates all the way to the root node which is then deleted
  - Since the total number of fusion operations are bounded by the height of the 2-4 tree that is O(log(n)), the total time to perform deletion in a (2,4) tree is O(log(n))

Performance of (2,4) tree

- The height of a $(2,4)$ tree storing n entries is $O(\log n)$
- A split, transfer, or fusion operation takes $O(1)$ time.
- A search, insertion, or removal of an entry visits $O(\log n)$ nodes.

Space utilization is always 50% or above.

B-Tree vs BST/Binary Search

- BST/Binary Search complexity if O(log(n))
- B-Tree Search complexity is $O(\log_d(n))$ => O(log(n)/log(d))

Book Reading

- Goodrich
  - B-Trees, Chapter 15, pages 697-714
  - (2,4) Trees, Chapter 11 section 11.5, pages: 502-511
- Adam Drozdek
  - Multiway Trees, Chapter 7 pages: 309-359

Next Lecture

- Lecture 12